```
      model:  supportedModel,
      system: "I'm happy to assist you in any way I can.
How can I be of service today?",
      messages: history,
    });

    for await (const text of textStream) {
      stream.update(text);
    }

    stream.done();
  })();

  return {
    messages: history,
    newMessage: stream.value,
  };
}
```

Iterates over the text stream from the AI model, updating our streamable value with each new piece of text

Returns an object containing the conversation history and the new message as a streamable value, allowing for real-time updates on the client

This server action encapsulates the entire process of interacting with the supported AI model, generating a response, and preparing it for streaming to the client. The `createStreamableValue` function is a helper function tool provided by the Vercel AI SDK for RSCs. It creates a special type of value that can be continuously updated on the server and streamed to the client in real time. This is particularly useful when dealing with AI-generated content that is produced incrementally.

The `continueConversation` function takes the conversation history as an input, sends it to the AI model, and then streams the response back. It uses the familiar `streamText` function that we explored in chapter 3. This approach allows for a more responsive user experience, as the AI's response can be displayed to the user as it's being generated, rather than waiting for the entire response to be completed.

One key line that you'll notice in the code is the presence of `'use server';` at the top of the file. This directive is not there by mistake or mere convention—it serves a key purpose in the RSC architecture.

The `'use server';` directive is a special instruction to the RSC compiler. When this directive is present, it marks the entire file to be used exclusively on the server. All functions and code within this file will only be executed on the server. This allows you to perform operations that require server resources, access databases, or interact with external APIs without exposing these operations to the client. The client (i.e., the browser) will never see or have access to this code. This is a significant security feature, as it allows you to keep sensitive logic, API keys, and other confidential information safely on the server.

It's worth noting that you can also use the `'use server'` directive on individual functions within a file, allowing for more granular control over which functions are server-only. However, when used at the top of a file, it applies to all exports from that file.

In the context of our AI-powered application, this directive ensures that all the logic for interacting with the OpenAI API, handling sensitive data, and managing the streaming of responses happens exclusively on the server. This not only enhances security but also allows for more efficient processing and improved performance.